Lecture 9: Indexing Structures for Files

EAS 550 — Database Systems and Query Language

1. Index Overview
2. B-Trees & B^+-Trees
3. B^+-Tree
4. Cost Model & Practice
5. Advanced Index Types
6. Physical DB Design

Lecture Roadmap

1. Index Overview

Why indexes? Insertion problem, I/O cost model, book analogy, dense/sparse, clustered vs. unclustered, SQL syntax.

2. B-Trees & B^+-Trees

Why multi-level? B-tree structure, B^+-tree node layout, why B^+-trees dominate, search algorithm, I/O count.

3. B^+-Tree Walkthrough

Complete 10-step insert trace (order $p = 3$), copy-up vs. push-up, range query, deletion.

4. Cost Model & Practice

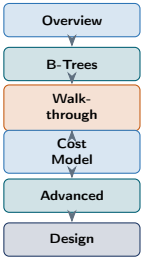
Height $h = \lceil \log_f(N/F) \rceil$, baseline cost $h+1$ I/Os, buffer caching reduces to $(h - L_B) + 1$, in-class exercise.

5. Advanced Index Types

Hash index (equality), bitmap (AND/OR), inverted index (text), composite and function-based indexes.

6. Physical DB Design

80-20 rule, when to index, which columns, clustered vs. unclustered, tuning cycle, bridge to hashing.



Why Indexes? — Motivation

Making retrieval efficient without reading every record

Setting: `Person(ssn, name, age)`, 1,000,000 rows, 100 rows/block.

Query: `SELECT * FROM Person WHERE age = 25`

A: Full Scan

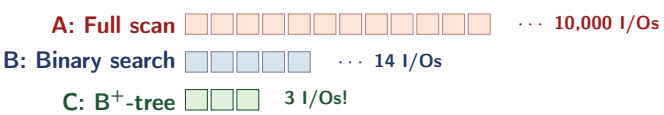
Read every block.
$10^6/100 = 10{,}000$ I/Os
OK only for tiny tables.

B: Binary Search

Data sorted by age.
$\lceil \log_2 10{,}000 \rceil = 14$ I/Os
One sort order only; insertions $O(N/P)$.

C: B$^+$-Tree

Fanout $f \approx 200$.
$h = \lceil \log_{200} 10{,}000 \rceil = 2$
$h + 1 = 3$ I/Os total!



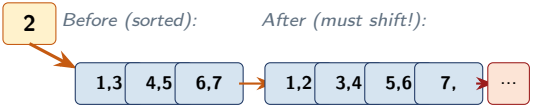
Each □ = one disk block read (10 ms). Root + internal + leaf + 1 data block.

Key trade-off

Faster reads (SELECT, JOIN) vs. **slower writes** (INSERT/UPDATE/DELETE must update every index). Optimizer decides *whether* to use an index.

The Insertion Problem — Why Not Just Sort?

Problem: insert age 2 into a sorted file.



Up to $\sim 2N/P$ I/Os to shift all blocks!

The fundamental tension

Sorting gives fast search ($O(\log N)$) but makes insertions expensive ($O(N/P)$ I/Os) — you must shift records to maintain order.

Can we get fast search *and* fast insertions at the same time?

Second problem: multiple attributes

You can only physically sort by *one* attribute. What if queries filter on `name`, `dept`, and `salary`?

Sorted by age

`WHERE age = 25` ✓ fast

`WHERE name = 'Ali'`

→ *full scan needed*

`WHERE salary > 80000`

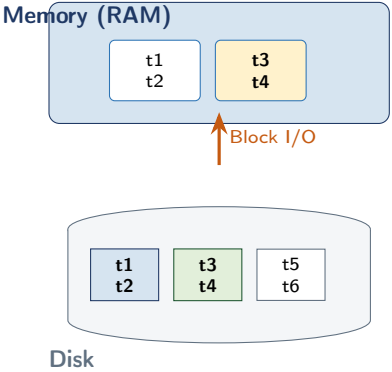
→ *full scan needed*

Solution: Indexes

Build *separate* small data structures (indexes) on each attribute. No need to sort the data file itself. Multiple indexes can coexist.

Quick Recap: Disk I/O Cost Model

Why we count block reads, not CPU operations



Why I/Os dominate cost

Disk read = **10 ms**; RAM access = **100 ns**.
Disk is 100,000× slower than RAM.
All tuples in a *block* come to memory together.
Cost metric = **number of blocks read**.

Consequence for indexes

An index lets us *skip* most blocks. Instead of reading all 10,000 blocks, we read 3 index blocks + 1 data block = **4 I/Os total**.

Speed-up: 2500× **faster!**

What Is an Index?

The same idea as a book index — applied to database records

Book index

Textbook

. . . content. . .
. . . content. . .

Index (back)

→

Dijkstra → **p. 23, 45**

Hashing → **p. 67, 89**

Indexing → **p. 12, 100**

SQL → **p. 5, 34**

Keyword ⟶ *page numbers*

Database index

Employee table

111 | **Alice**
222 | **Bob**
333 | **Carol**
444 | **Dave**

→

Index on Ssn

111 → **Blk 2**
222 → **Blk 5**
333 → **Blk 1**
444 → **Blk 8**

Key value ⟶ *disk block address*

In one sentence

An **index** maps *search-key values* to the *disk blocks* containing matching records, without changing how the data file is stored.

Key trade-off: faster reads vs. slower writes

Every INSERT/UPDATE/DELETE must also update *all* indexes. The optimizer decides whether to *use* a given index.

Index Types — Landscape View

Different workloads need different indexes

Index Type Landscape

B^+-Tree (default)

Range + equality
Sorted data
Default in all RDBMS

✓ *PostgreSQL, MySQL, Oracle*

Hash Index

Equality only
O(1) point lookup
Static or dynamic

PostgreSQL USING HASH

Bitmap Index

Low-cardinality
AND/OR queries
Data warehouses

Oracle, Redshift

Specialty

Full-text, GIS,
Vector (HNSW),
Time-series

Various extensions

This lecture covers: B^+-Tree (deep dive + walkthrough) Hash Index Bitmap Index Composite / Function-based indexes

Rule of thumb

Default to B^+-tree. Use Hash only when queries are purely equality. Use Bitmap on low-cardinality warehouse columns. Everything else is special-purpose.

Query → Index map

`WHERE k = v`	B^+ or Hash
`WHERE k BETWEEN a AND b`	B^+ only
`WHERE dept = 'HR'`	Bitmap
`ORDER BY k`	Clustered B^+

Types of Single-Level Ordered Indexes

Type	Ordering field	Field type	Dense?
Primary Index	Ordering **key** field of a sequentially ordered file	Distinct value per record (key)	Sparse
Clustering Index	Ordering **non-key** field	Repeated values allowed (non-key)	Sparse
Secondary Index	Any non-ordering field	Key or non-key	Dense

Primary Index

One entry per *data block*.
Entry = (first key in block, block ptr).
Possible only when file is ordered by that key.

Clustering Index

One entry per *distinct value* of the ordering non-key field.
Groups of records with same value are stored contiguously.

Secondary Index

Provides access on a non-ordering field.
Must be *dense*: one entry per record (or indirection via buckets).
Multiple allowed per table.

Index Types in Action — Student Database

One table – primary index (SID), clustering index (Major), secondary index (GPA)

Student table

(file sorted by SID)

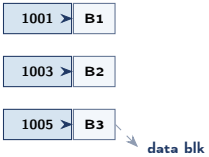
SID	Name	Maj	GPA	
1001	Alice	CS	3.8	B1
1002	Bob	CS	3.3	B1
1003	Carol	EE	2.9	B2
1004	Dave	EE	3.1	B2
1005	Eve	ME	3.9	B3
1006	Frank	ME	2.7	B3

Blk = disk block

Primary Index

on SID (ordering key)

Sparse — 1 entry/block



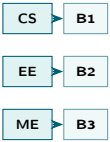
Query: SID

$1001 \leq 1003 < 1005$
→ fetch B2: Carol, Dave.
2 I/Os total.

Clustering Index

on Major (non-key)

Sparse — 1 entry/value



all CS rows contiguous in B1

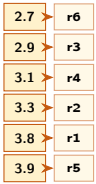
Query: Major

→ entry EE → B2.
Fetch B2: Carol, Dave.
2 I/Os (clustered!).

Secondary Index

on GPA (non-ordering)

Dense — 1 entry/record



Query: GPA≥3.8

→ 3.8→r1, 3.9→r5.
Must be **dense**: GPA not ordering field.

Dense Index vs. Sparse Index

Dense Index (1 entry per *record*)

Index	Data records
10	rec10
20	rec20
30	rec30
40	rec40
50	rec50
60	rec60

✓ Works on unsorted data; usable as secondary index

— N entries — larger index file

Sparse Index (1 entry per *block*)

Requires data sorted by search key.

Index	Data blocks
10	10,20,30
40	40,50,60
70	70,80,90

✓ Only $\lceil N/P \rceil$ entries; may fit in memory

— Requires sorted (clustered) data

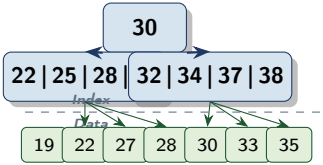
Lookup $K = 50$: anchor 40 $\rightarrow$ block $\{40, 50, 60\}$. Scan. **2 I/Os.**

Sparse index requires clustered data

Works *only* if data is sorted by search key. Consequence: **sparse** $\Leftrightarrow$ **clustered**; unclustered tables must use dense indexes.

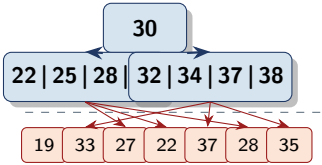
Clustered vs. Unclustered Index

✓ Clustered Index



Same order as index

× Unclustered Index



Each ptr → different block

Range query cost

≈ 1 seek $+ \lceil R/P \rceil$ sequential reads.
Records contiguous on disk.

Range query cost

Up to R random seeks. For $R = 100{,}000$:
$100{,}000 \times 10\text{ms} = 17$ min!

Rule: at most ONE clustered index per table

Clustered ⇒ fast range queries. Unclustered ⇒ fine for point lookups, expensive for ranges. PK is typically clustered.

How an Index Works — Point Lookup

B$^+$-tree lookup: root → internal → leaf → data = $h + 1$ block reads

```
SELECT *
FROM   Movies
WHERE  title = 'Ponyo'
  AND  year  = 2008;
```

How it works

Traverse the B$^+$-tree reading **one block per level**. At the leaf a pointer gives the exact data block address.

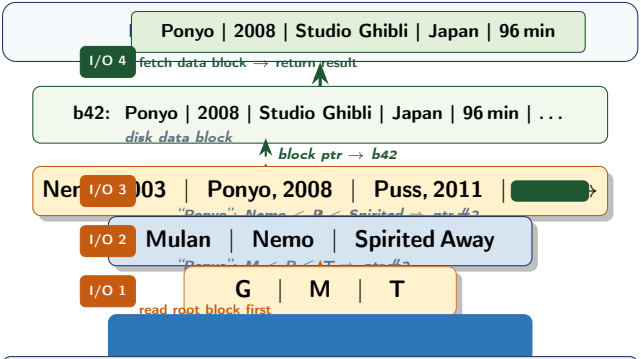
Cost: h

3 index I/Os + 1 data I/O = 4

Full scan: 10,000 I/Os

Speed-up: ×**2,500!**

General: $h + 1$ total I/Os.



3 index I/Os + 1 data I/O = 4 total vs. **10,000 I/Os** without index

Indexes Dramatically Accelerate Joins

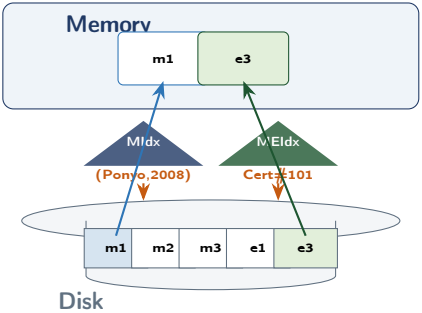
```
CREATE INDEX MIdx  ON Movies(title,year);
CREATE INDEX MEIdx ON MovieExec(cert_num);

SELECT E.name FROM Movies M, MovieExec E
WHERE M.title = 'Ponyo' AND M.year = 2008
  AND M.producerC = E.cert_num;
```

With both indexes

Step 1: MIdx → find Ponyo 2008 (≈3 I/Os).
Step 2: MEIdx → fetch exec (≈3 I/Os).
Total: ≈6 I/Os vs. $N_M \times N_E$ I/Os.

NL-join vs. index NL-join

Without: $1{,}000 \times 10{,}000 = 10^7$ comparisons.
With index: $1{,}000 \times \lceil \log_f N_E \rceil \approx 4{,}000$ I/Os.
Indexes matter more for joins than point lookups.



Creating and Dropping Indexes in SQL

General form

```
CREATE [UNIQUE] [CLUSTER] INDEX idx_name
  ON table_name
     (col1 [ASC|DESC], col2, ...);

DROP INDEX idx_name;
```

Examples

```
CREATE INDEX idx_person_age
  ON Person(age);              -- B+-tree,
      default

CREATE INDEX idx_emp_dept_age
  ON Employee(Dno, Age);       -- composite

CREATE UNIQUE INDEX idx_emp_ssn
  ON Employee(Ssn);            -- enforces
      uniqueness

CREATE INDEX idx_status
  ON Orders(status) USING HASH;
```

Four rules to remember

1. `UNIQUE`: DBMS rejects duplicate key inserts
2. `CLUSTER`: physically reorders data file; **one per table**
3. Every index **slows writes**: all indexes updated on INSERT/UPDATE/DELETE
4. **Optimizer** decides whether to use an index; `CREATE` only makes it *available*

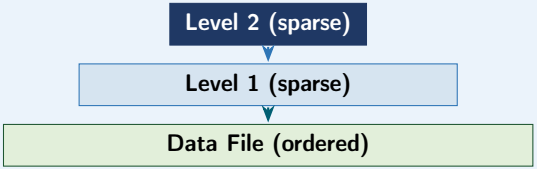
Composite index prefix rule

Index on (A_1, A_2): useful for A_1 alone, or A_1+A_2 together. Not efficient for A_2 alone.

Multi-Level Indexes: Motivation for B$^+$-Trees

Multi-level index idea

When the first-level (base) index is still too large to fit in memory, build a *sparse index on the index itself*. Repeat until the top level fits in memory.



Problem with static multi-level indexes (ISAM): Fixed structure — insertions may cause overflow chains, degrading performance.

B$^+$-Tree: dynamic & balanced

Formalises multi-level indexing with two guarantees:

(1) Balanced: all leaf nodes at same depth.
(2) At least half-full: each node has $\geq \lceil p/2 \rceil$ children, preventing wasted space.

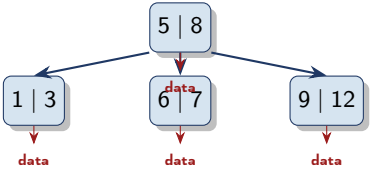
Result: $h \leq \lceil \log_f N \rceil$ where $f = \lceil p/2 \rceil$ is the minimum fanout.

Fan-out f	Height for 10^6 recs	I/Os
10	6	7
50	4	5
100	3	4
200	3	4

A node in a B-tree of order p:

Example: B-tree of order $p = 3$

Keys: 8, 5, 1, 7, 3, 12, 9, 6



B-tree rules (order p)

Max keys/node: $p - 1$; max tree ptrs: p
Min keys (non-root): $\lceil p/2 \rceil - 1$
Data ptrs Pr_i: in *every* node (internal + leaf)

Problem: bulky internals

Data pointers in every node $\Rightarrow$ fewer keys per node $\Rightarrow$ **lower fan-out** $\Rightarrow$ taller tree.

B^+-tree fix

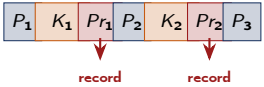
Data ptrs only at leaf nodes. Internal nodes: keys + tree ptrs $\Rightarrow$ **much higher fan-out** $\Rightarrow$ shorter tree $\Rightarrow$ fewer I/Os.

B-Tree vs. B$^+$-Tree — Side-by-Side

The critical difference: data pointers in every node vs. leaves only

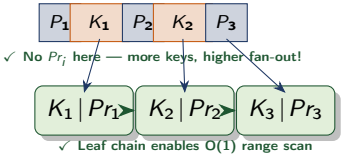
Feature	B-Tree	B$^+$-Tree
Data ptrs in **internal** nodes	**Yes**	**No**
Data ptrs in **leaf** nodes	Yes	Yes
Leaf nodes linked	**No**	**Yes**
Fan-out per node	Lower	**Higher**
Tree height (same N)	Higher	**Lower**
Range queries	Harder	**Easy**
Used in RDBMS	Rare	**Default**

B-tree internal node — data ptrs *bloat* every node:



Fewer keys/node ⇒ lower fan-out ⇒ taller tree

B$^+$-tree — lean internals + linked leaf chain:



Why B^+-Trees Dominate Databases

Three structural guarantees that make them the universal default

1. High fanout $\Rightarrow$ shallow tree

Internal nodes store *only* keys + pointers (no data). A 4 KB page holds $\approx$ 100–200 keys.

Result: even 10^9 records need a tree of height ≤ 5.

Impact: every lookup costs $\leq$ 6 disk reads.

2. Linked leaves $\Rightarrow$ fast range scans

Leaf nodes are chained as a sorted linked list.

Range query: find the lower bound once (h I/Os), then follow P_{next} pointers linearly.

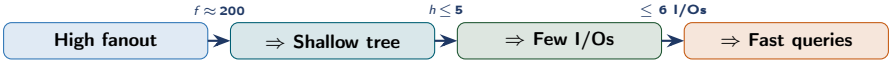
No need to re-traverse the tree for each successive result.

3. Balanced $\Rightarrow$ predictable performance

The tree can only grow upward (root splits). Every leaf is at the same depth.

No path from root to leaf is ever longer than another.

Worst-case = average-case = $h + 1$ I/Os.



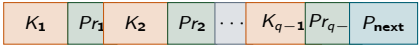
B^+-Tree: Internal Node vs. Leaf Node

Data pointers appear ONLY at leaf nodes

(a) Internal node (guides search; no data pointers):

P=child ptr; K=search key

(b) Leaf node (holds data pointers + next-leaf link):



Pr=record ptr; P_{next}=next-leaf ptr

Key advantages over B-tree

Higher fan-out: internal nodes store *only* keys and tree ptrs — more keys fit per page.

Range scans: once first qualifying leaf found, follow P_{next} chain without re-traversing.

Uniform cost: every access goes through a leaf, giving predictable $O(\log_p N)$ I/Os.

Note on key duplication

Keys in internal nodes are *copies* (routing only); actual key-record pair lives in a leaf. Hence "copy-up" (not "push-up") for leaf splits.

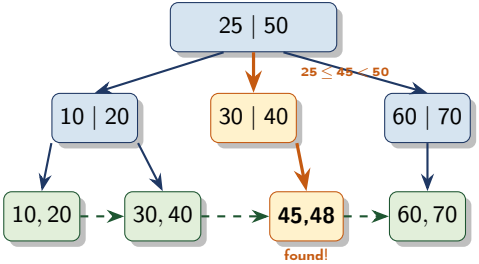
B$^+$-Tree Search Algorithm

Search for key K

```
procedure BplusSearch(root, K):
  node ← root
  while node is an internal node:
    if K < node.K[1]:
      node ← node.P[1]
    elif K ≥ node.K[q-1]:
      node ← node.P[q]
    else:
      find i s.t. K[i] ≤ K < K[i+1]
      node ← node.P[i+1]
  // Now at a leaf node
  if K in node.keys:
    return node.Pr[K]  // record ptr
  else:
    return NOT_FOUND
```

I/O cost: height h = number of nodes read on path from root to leaf + 1 data block read.

Example: search for $K = 45$



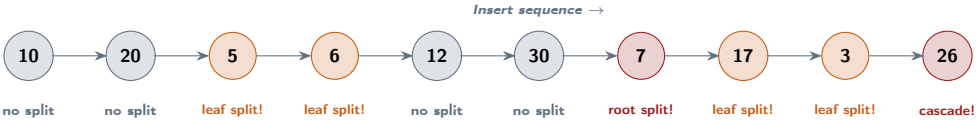
I/O count

Root(1) → Internal(1) → Leaf(1) → Data(1) = **4 I/Os**

Same cost for any table size at height 3!

B^+-Tree Complete Walkthrough: Setup

Order $p = 3$, max 2 keys/node, inserting sequence: 10, 20, 5, 6, 12, 30, 7, 17, 3, 26



Rules for order p

Max keys/node: 2 **Min keys (non-root):** 1

Leaf split (copy-up):
Overflow $[A, B, C]$: keep $[A, B]$ left, move $[C]$ right, *copy* C up to parent. C stays in leaf.

Internal split (push-up):
Overflow $[K_1, K_2, K_3]$: left keeps K_1, right gets K_3, *push* K_2 up. K_2 leaves both children.

Copy-up vs. Push-up — the key distinction

Leaf split	Promoted key *stays* in right leaf (it's a routing copy)
Internal split	Promoted key is *removed* from both children (pushed upward)

Colour key for diagrams

internal node leaf node just split

After insert 10:

10

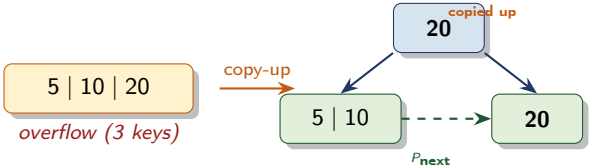
After insert 20:

10 | 20

Insert 5: leaf $[10, 20]$ becomes $[5, 10, 20]$ — overflow!

Leaf split — three keys overflow

Keep $[5, 10]$ in left leaf; move $[20]$ to new right leaf; **copy** 20 up to (new) root.

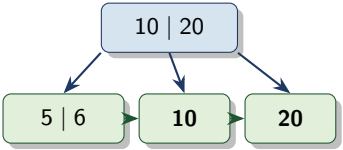


Why 20 and not 10?

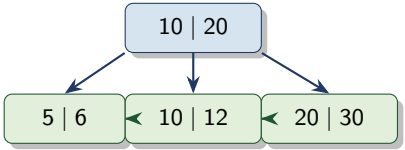
We keep $[A, B]$ in the left leaf and move $[C]$ to the right; C is the first (smallest) key of the right leaf, so it is the value copied up. Here A=5, B=10, C=20.

Walkthrough Steps 4–6: Insert 6, 12, 30

Insert 6: $6 < 20 \rightarrow$ left leaf $[5, 10] \rightarrow [5, 6, 10]$
Overflow! Keep $[5, 6]$, move $[10]$ right, copy 10 up.
Root $[20] \rightarrow [10, 20]$. Still fits. ✓



Insert 12: $10 \leq 12 < 20 \rightarrow$ leaf $[10] \rightarrow [10, 12]$. No overflow.
Insert 30: $30 \geq 20 \rightarrow$ leaf $[20] \rightarrow [20, 30]$. No overflow.

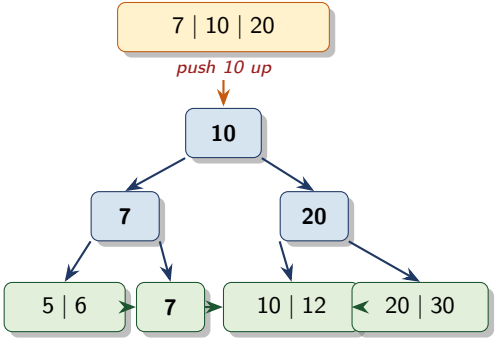


State after steps 4–6

Root: $[10, 20]$ Leaves: $[5, 6] \rightarrow [10, 12] \rightarrow [20, 30]$
Root is still valid: 2 keys, 3 children. Tree height = 2.

Insert 7: $7 < 10 \rightarrow$ left leaf $[5, 6] \rightarrow [5, 6, 7]$ — overflow!
Keep $[5, 6]$, move $[7]$ right, copy 7 up to root $[10, 20] \rightarrow [7, 10, 20]$ — **root overflow**!

Root split (push-up):
Root $[7, 10, 20]$: left gets $[7]$, right gets $[20]$, **push** 10 up to a new root.



Push-up vs. Copy-up recap

Leaf split $\rightarrow$ *copy-up*: promoted key **stays** in the right leaf.
Internal split $\rightarrow$ *push-up*: promoted key is **removed** from the internal node — lives *only* in the parent.
Here: 10 no longer appears in either child.

Only way to grow taller

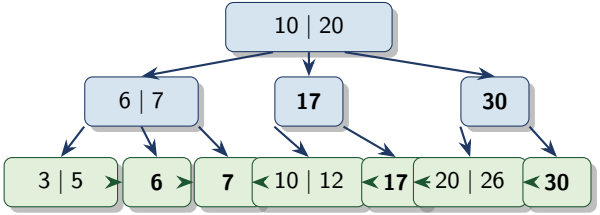
A B^+-tree grows in height *only* when the root splits. This ensures all leaves remain at the same depth — the **balance invariant**.
Tree is now height 3.

Insert 17: $17 \geq 10 \rightarrow$ right subtree, $17 < 20 \rightarrow$ leaf $[10, 12] \rightarrow [10, 12, 17]$.
Split: keep $[10, 12]$, move $[17]$; copy 17 up.
Right internal $[20] \rightarrow [17, 20]$. ✓

Insert 3: $3 < 10 \rightarrow$ left subtree, $3 < 7 \rightarrow$ leaf $[5, 6] \rightarrow [3, 5, 6]$.
Overflow! Keep $[3, 5]$, move $[6]$; copy 6 up.
Left internal $[7] \rightarrow [6, 7]$. ✓

Insert 26: $26 \geq 10 \rightarrow$ right subtree, $26 \geq 20 \rightarrow$ leaf $[20, 30] \rightarrow [20, 26, 30]$.
Overflow! Keep $[20, 26]$, move $[30]$; copy 30 up.
Right internal $[17, 20] \rightarrow [17, 20, 30]$ — **overflow!**
Split: keep $[17]$, move $[30]$; push 20 up.
Root $[10] \rightarrow [10, 20]$. ✓

Final tree after all 10 insertions:



Range Query on the Final Tree Using the leaf chain for range: `SELECT * FROM T WHERE key BETWEEN 7 AND 20`

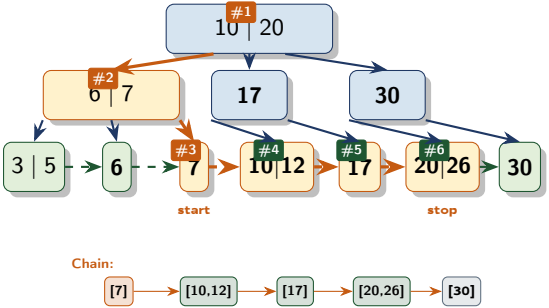
Step 1 — Locate lower bound $K = 7$:

1. Root $[10, 20]$: $7 < 10 \Rightarrow$ go left (I/O 1)
2. Internal $[6, 7]$: $7 \geq 7 \Rightarrow$ go right (I/O 2)
3. Leaf $[7]$: key 7 found (I/O 3)

Step 2 — Scan via P_{next} chain:

1. Leaf $[7]$: collect **7** (already loaded)
2. Leaf $[10, 12]$: collect **10, 12** (I/O 4)
3. Leaf $[17]$: collect **17** (I/O 5)
4. Leaf $[20, 26]$: collect **20**; stop when key > 20 (I/O 6)

Result: $\{7, 10, 12, 17, 20\}$ in 6 I/Os total.



B^+-Tree Deletion — Three Cases

Maintain: every non-root leaf has $\geq \lceil (p-1)/2 \rceil = 1$ key

Case 1: No underflow

Delete $K = 12$.
Leaf $[10, 12] \rightarrow [10]$.
$[10]$ has 1 key $\geq$ min. ✓
Done; parent routing keys unchanged.

Case 2: Borrow from sibling

Delete $K = 10$ from $[10, 12]$.
$[10, 12] \rightarrow [12]$: still valid ($\geq 1$). But if result has 0 keys: Borrow the largest key from left sibling (or smallest from right) and update the parent routing key.

Case 3: Merge

Both the underflowing leaf and its sibling have exactly the minimum.
Merge the two leaves into one.
Remove the corresponding separator key from the parent.
Parent may underflow $\Rightarrow$ recurse upward.

Also: update routing keys

If the *smallest key* of a leaf changes (e.g. the minimum key was deleted), the corresponding routing key in an ancestor internal node must also be updated. This is a subtle correctness issue.

Deploying B$^+$-Tree Indexes in PostgreSQL

Create table + indexes

```sql
CREATE TABLE orders (
  order_id     SERIAL PRIMARY KEY,
  customer_id  INT    NOT NULL,
  order_date   DATE   NOT NULL,
  status       VARCHAR(20),
  total        NUMERIC(10,2)
);
CREATE INDEX idx_cust
  ON orders(customer_id);
CREATE INDEX idx_date
  ON orders(order_date DESC);
CREATE INDEX idx_comp
  ON orders(customer_id, order_date);
```

Verify with EXPLAIN ANALYZE

```sql
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 42;
-- Index Scan using idx_cust

EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_date BETWEEN
  '2024-01-01' AND '2024-03-31';
-- Index Scan using idx_date
-- (leaf chain used for range)
```

Index choice per query

Q1 `WHERE cust_id=?` → `idx_cust`
Q2 `WHERE order_date BETWEEN` → `idx_date`
Q3 `WHERE status=?` → Bitmap preferred
Q4 `WHERE order_id=?` → PK index

B^+-Tree Cost Model — Tree Height

How tall is the tree?

Model variables

N	data pages to index
f	fanout (avg. children per node)
F	fill factor $\in (0, 1]$; typical $\frac{2}{3}$

Tree height formula

$h = \lceil \log_f(N/F) \rceil$ Large $f \Rightarrow$ shallow tree.
$F < 1$ reserves slack for faster insertions.

Baseline I/O cost (no caching)

$\text{IO}_{\text{point}} = h + 1$ (h index reads + 1 data read)
Example: $f = 200$, $N = 10{,}000$:
$h = \lceil \log_{200}(10{,}000) \rceil = 2 \Rightarrow 3$ I/Os total.

Height vs. fanout ($N = 10^6$ recs)

Fanout f	Height h	I/Os $(h+1)$
$f = 10$	$h = 6$	7
$f = 50$	$h = 4$	5
$f = 100$	$h = 3$	4
$f = 200$	$h = 3$	4
$f = 500$	$h = 2$	3

High fanout $\Rightarrow$ shallow tree $\Rightarrow$ fewer I/Os.

1 billion records

$f = 100$, $N = 10^9$: $h = \lceil \log_{100}(10^9) \rceil = 5$. Cost $= 5 + 1 = 6$ I/Os. *A whole RDBMS, in 6 reads.*

B^+-Tree Cost Model — Buffer Pool Effect

Caching top tree levels reduces disk I/Os dramatically

Buffer pool variables

B	buffer pages for top tree levels
L_B	levels fitting in B: $B \geq \sum_{l=0}^{L_B-1} f^l$

I/O cost with buffer pool

Point: $\text{IO}_{\textbf{point}} = (h - L_B) + 1$

Range $[a, b]$: $\text{IO}_{\textbf{range}} = (h - L_B) + \text{Cost(OUT)}$

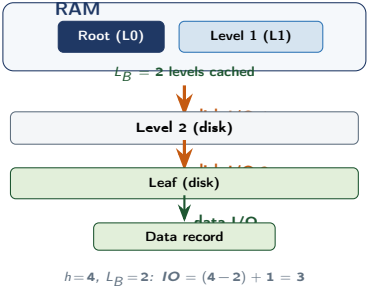
Cost(OUT) = leaf pages + data pages fetched.

Key intuition

Root + level 1 live in RAM permanently.

Most point queries: only 1–2 disk I/Os.

$h + 1$ baseline shrinks to the uncached tail only.



Exercise preview

$h = 5$, $f = 10$, $B = 11 \Rightarrow L_B = 2$; IO$= (5 - 2) + 1 = 4$.

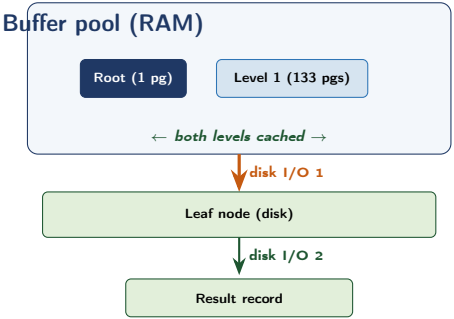
B^+-Trees in Practice — Typical Parameters

Why real databases need only 1–2 disk I/Os per point lookup

Typical real-world values

Degree d	≈ 100
Fill factor	67% (average)
Avg. fanout f	≈ 133

Level	Pages	Size
0 (root)	1	8 KB
1	133	1 MB
2	17,689	133 MB
3 (leaves)	2,352,637	17.8 GB



"Aha" moment

Full cost $= h + 1$ I/Os total. With root + level 1 cached, only the **leaf + data page** need disk reads. For a **17.8 GB** relation: typically just **2 disk I/Os per point query**.

In-Class Exercise: Cost Model

Given

N = 100,000 data pages
F = 1 (no fill slack)
f = 10 (fanout)
B = 11 buffer pages

Find

(1) Tree height h
(2) Cached levels L_B
(3) I/O cost for a point query

Solution

Step 1 — height:
$h = \lceil \log_{10}(100{,}000) \rceil = 5$

Step 2 — cached levels L_B:
$L_B = 2$: $1 + 10 = 11 \leq B = 11$. ✓
$L_B = 3$: $1 + 10 + 100 = 111 > 11$. ×
$\therefore L_B = 2$.

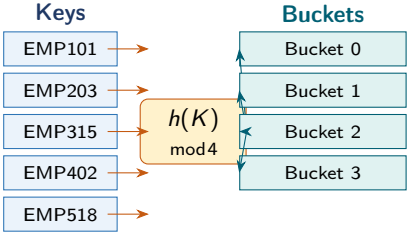
Step 3 — I/O cost:
$\text{IO} = h - L_B + 1 = 5 - 2 + 1 = 4$

Interpretation

Root + first level (11 pages total) live in the buffer.
Remaining 3 levels must be fetched from disk:
Level 2 (1 I/O) → Level 3 (1 I/O) → Level 4 (1 I/O) → Level 5 = leaf (1 I/O).
Total: **4 disk reads**.

Hash index

Applies $h(K)$ mod M to search key K to identify a *bucket* pointer. Entry: (K, Pr) where Pr is a record pointer. Avg. $O(1)$ lookup for equality queries.



Use hash when

- Equality-only queries (`WHERE K = v`)
- Data is relatively static
- High cardinality key field

Do NOT use hash when

- Range queries (`BETWEEN`, $<$, $>$)
- `ORDER BY` on indexed column
- Frequently inserted/deleted data

Bitmap index

For each distinct value v, a bit array $B_v[i] = 1$ iff record i has value v. Multi-condition queries use fast bitwise AND/OR.

ID	Dept	Gender
1	Sales	M
2	HR	F
3	Sales	F
4	IT	M
5	HR	M
6	IT	F
7	Sales	F

	Dept			Gender	
ID	S	H	I	M	F
1	1	0	0	1	0
2	0	1	0	0	1
3	1	0	0	0	1
4	0	0	1	1	0
5	0	1	0	1	0
6	0	0	1	0	1
7	1	0	0	0	1

```
WHERE Dept='Sales' AND Gender='F'
```

Sales=1010001 AND F=0110011 = 0010001 ⇒ **rows 3,7**. One bitwise AND!

Inverted Index — Full-Text Search

Same non-clustered structure as a secondary index, applied to text terms

Inverted index

Maps each **term** (word or token) to the list of document IDs (and optionally positions) where the term appears.

Structurally identical to a secondary B^+-tree index: **key** = term, **pointer list** = document IDs.

Term	Posting list (doc IDs)
cat	doc3, doc8, doc21
database	doc1, doc4, doc7
dog	doc1, doc4
index	doc2, doc4, doc9
query	doc1, doc6

Multi-term AND: "database index"

`database` $\rightarrow \{1, 4, 7\}$; `index` $\rightarrow \{2, 4, 9\}$
Intersect: $\{1, 4, 7\} \cap \{2, 4, 9\} = \{4\}$
Only doc 4 matches. No disk reads until result known.

Posting list metadata options

Basic	doc ID only
TF-IDF	term frequency per doc
Positional	(doc, pos) for phrase queries
Field-typed	title vs. body weights

Real systems

Elasticsearch, PostgreSQL `GIN`, Oracle Text, SQLite FTS5.
Structurally identical to a secondary B^+-tree: key = term, pointer list = doc IDs.

Multi-Key Indexes and Function-Based Indexes

Composite (multi-key) index:

```sql
-- Index on (Dno, Age) combined
CREATE INDEX idx_emp
  ON Employee(Dno, Age);

-- Uses index (Dno prefix present):
SELECT * FROM Employee
  WHERE Dno=5 AND Age=30;

-- Uses index (Dno prefix only):
SELECT * FROM Employee
  WHERE Dno=5;

-- Does NOT efficiently use index:
SELECT * FROM Employee
  WHERE Age=30;  -- no Dno prefix!
```

Prefix rule

A composite index $(A_1, A_2, \ldots)$ is only useful when the leftmost attribute(s) appear in the WHERE clause.

Function-based index:

```sql
-- Index on computed value
CREATE INDEX idx_upper_lname
  ON Employee(UPPER(Lname));

-- This query NOW uses the index:
SELECT * FROM Employee
  WHERE UPPER(Lname) = 'SMITH';

-- Without it: UPPER() prevents index
-- use even if Lname is indexed.
```

Why it matters

Without a function-based index, applying any function to an indexed column (e.g. `UPPER`, `EXTRACT`, `SUBSTR`) prevents the optimizer from using the index, forcing a full scan.

Index Type Quick Reference

Index Type	Best for	Point Q	Range Q	Updates	Where used
B^+-tree (clustered)	Range, ORDER BY	$O(\log_f N)$	**Excellent**	Moderate	PG, MySQL, Oracle
B^+-tree (unclustered)	Secondary access paths	$O(\log_f N)$	OK	Moderate	Multiple per table
Hash index	Equality-only lookups	**O(1) avg**	**None**	Costly on resize	PG `USING HASH`
Bitmap index	Low-cardinality columns	Fast (bitwise)	AND/OR ops	Very costly	Oracle, Redshift DW
Inverted index	Full-text / keyword search	Fast	Term union	Costly rebuild	Elasticsearch, PG GIN

Rule of thumb

Default: B^+-tree. Hash: equality-only static data. Bitmap: <100 distinct values. Inverted: free-text.

Query → index decision

`WHERE k = v`	B^+ or Hash
`WHERE k BETWEEN a AND b`	B^+ only
`WHERE dept = 'HR'`	Bitmap
`CONTAINS 'cat'`	Inverted
`ORDER BY k`	Clustered B^+

80-20 Rule (Pareto Principle)

80% of database processing is caused by only 20% of queries. Focus index design on that critical 20%.

Decision	Guideline
Index at all?	Yes: attr in WHERE/JOIN, has SLA, is a key. No: tiny table; write-dominated; very low selectivity.
Which attribute(s)?	Single for simple queries; composite for multi-attr WHERE (obey prefix rule).
Clustered or not?	At most one per table. Pick most critical range-query attr (usually PK).
Hash vs. tree?	B^+-tree is the default (equality + range + sort). Use Hash only for equality-only, static data.

When NOT to index

- Table is small
- **Low cardinality**: `gender`, `is_active` — use Bitmap instead
- **Frequently updated** column — every write updates the index
- **Write-heavy** workloads where SELECT is rare

Index Selectivity — Knowing Which Columns Benefit Most

Not all columns are created equal

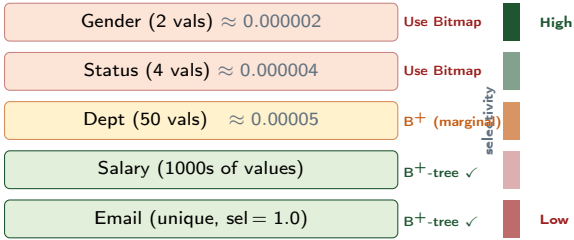
Selectivity (definition)

$$\text{sel}(C) = \frac{|\textbf{distinct values in } C|}{|\textbf{total rows}|}$$

High (near 1.0): many distinct values — index very useful.
Low (near 0): few distinct values — optimizer may prefer full scan.

Optimizer rule of thumb

If query matches > 5–10% of rows, clustered scan is often faster than following many random index pointers (unclustered).



Column cardinality in `Employee` table with 1,000,000 rows

Reasons to ADD an index

- Query does full table scan (`EXPLAIN` shows Seq Scan on large table)
- Attribute appears frequently in WHERE or JOIN (top-20% queries)
- A response-time SLA is violated
- Range query on frequently accessed attribute is slow

Reasons to REMOVE an index

- Never chosen by optimizer (`pg_stat_user_indexes.idx_scan = 0`)
- Attribute updated very frequently — maintenance cost exceeds benefit
- A broader composite index supersedes a narrower one
- Low selectivity (e.g. 2–4 values): use Bitmap instead

Practical tuning cycle

(1) Run `EXPLAIN ANALYZE` on the top-10 slow queries. **(2)** Add missing indexes; drop zero-use ones. **(3)** Rebuild fragmented indexes: `REINDEX TABLE t;`. **(4)** Re-measure. Repeat as data distribution evolves.

Bridge to Next Lecture: When B$^+$-Trees Aren't Enough

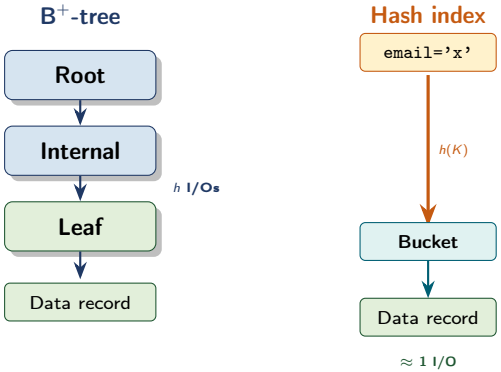
B$^+$-tree excels at

Range queries (`BETWEEN`, $<$, $>$), ordered scans (`ORDER BY`), prefix queries (`LIKE 'abc%'`). **Safe general-purpose default.**

Equality-only workloads

```
SELECT * FROM Sessions
WHERE session_id = 'abc123';
SELECT * FROM Users
WHERE email = 'alice@x.com';
```

B$^+$-tree: $O(\log_f N)$, traverses h levels.
Hash: goes **directly** to bucket, $O(1)$ avg.
But: hash $\neq$ range queries or ORDER BY.



Next lecture (Ch. 17.8)

Static hashing — fixed bucket array.
Extendible hashing — dynamic directory.
Linear hashing — incremental, no directory.

Summary — Index Quick Reference

Type	Dense?	Sorted req.	Point Q	Range Q	Max/table	Best for
Primary Index	Sparse	Yes (key)	Fast	Good	1	Ordered key lookup
Clustering Index	Sparse	Yes (non-key)	Fast	Good	1	Grouped records
Secondary Index	Dense	No	Fast	OK	Unlimited	Non-ordering fields
Clustered B^+	—	Yes	Fast	**Best**	1	General-purpose
Unclustered B^+	—	No	Fast	Costly	Unlimited	Secondary access
Hash Index	—	No	**O(1)**	**None**	Unlimited	Equality only
Bitmap	Bit-array	No	Fast	AND/OR	Many	Low cardinality

B^+-tree walkthrough recap

10 insertions $\rightarrow$ 6 leaf + 1 internal + 1 root split.
Final tree: height 3, root $[10, 20]$.
Range $[7, 20]$: 6 I/Os (3 tree + 3 leaf chain).

Next lecture: Hashing

Static, extendible, and linear hashing (Ch. 17.8).

Think about: when does hash beat B^+-tree, and when does it fail?

Reading: Ch. 17.8.